

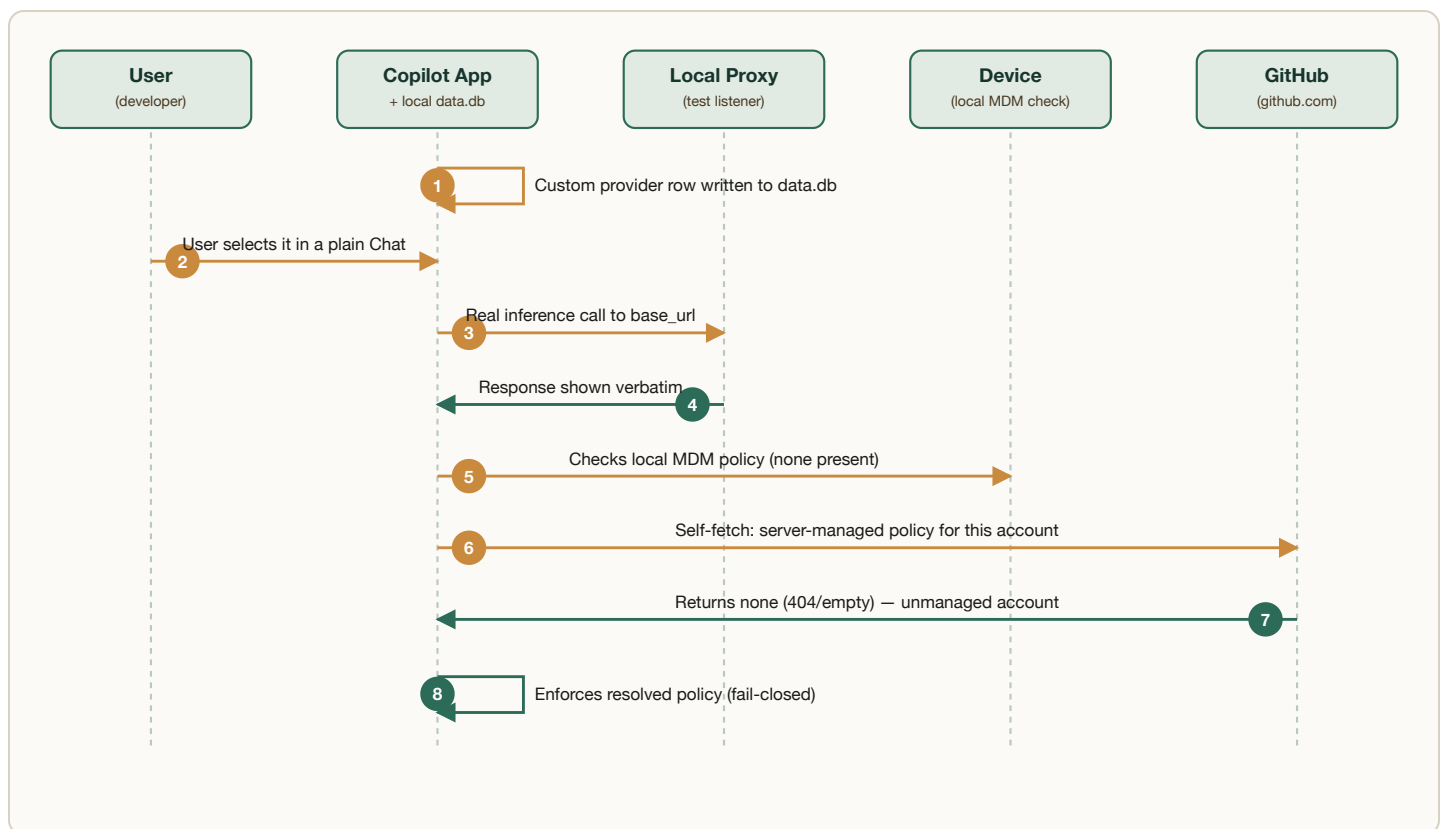
GitHub Copilot app: confirmed vs. still-to-confirm, end to end

Two diagrams: what's been directly proven (live tests + verbatim GitHub docs), and what's still needed to make an enterprise-scale, fleet-wide interception strategy actually work.

Diagram 1 — What's confirmed today

Every arrow here is either a live-tested behavior (the 2026-09-24 BYOK test) or a fact read verbatim from GitHub's own docs/logs — nothing inferred or assumed.

● Outbound / initiating ● Return / internal resolution



What each arrow means

- 1 Custom provider row written to `data.db` proven**

A real `type='custom'` row was inserted directly into a live install's SQLite database, with the app closed to avoid a write race. Confirmed to survive a relaunch, matching what was written byte for byte.
- 2 User selects it in a plain Chat proven**

Selecting the provider only works in a plain Chat. Project/agentive sessions lock to GitHub's own first-party model (observed: `mai-code-1.1-flash`) and never show a picker reaching a custom provider at all.
- 3 Real inference call to `base_url` proven**

The app's actual inference request — the full 126 KB payload, including GitHub's real internal system prompt — was observed hitting a local listener. Not a stripped-down or sandboxed call.

4 **Response shown verbatim** proven

The listener's fixed placeholder text appeared as the real chat response in the app UI, proving the app relays whatever this hop returns without alteration.

5 **Checks local MDM policy (none present)** proven

Caught live in the app's own running logs: `[managedSettings] device MDM: no policy present on this device` — a real, unprompted check against OS-level managed preferences.

6 **Self-fetch: server-managed policy for this account** proven

Also caught live: `self-fetch starting for account https://github.com/ukanitkar` — a genuine authenticated API call to GitHub for this specific signed-in account.

7 **Returns none (404/empty) — unmanaged account** proven

Live log: `server policy: none for this account (404/empty) from https://github.com`.

Expected result for a personal, non-enterprise-managed account — proves the fetch is real, not that the mechanism is inert.

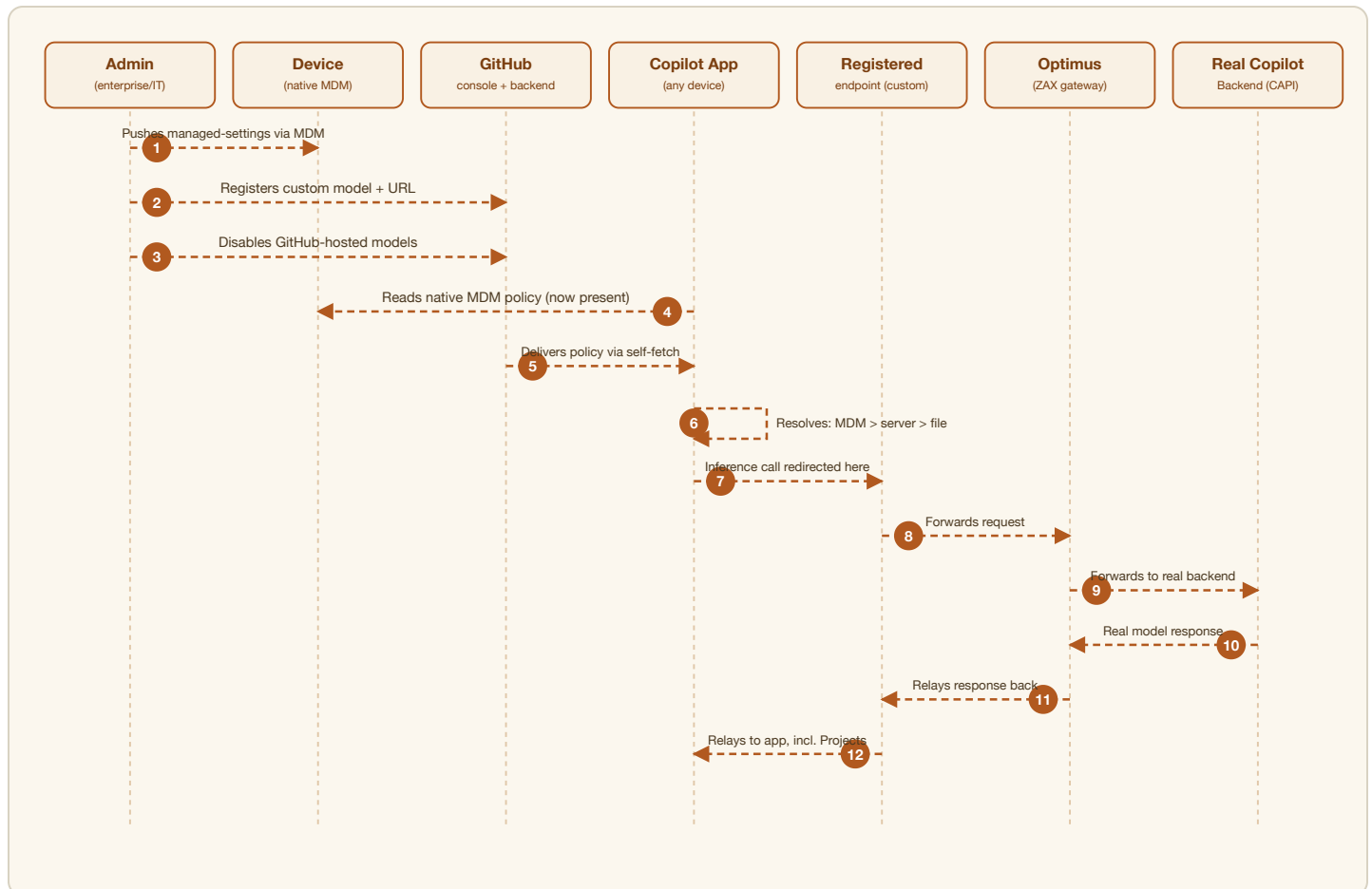
8 **Enforces resolved policy (fail-closed)** proven

With no policy from either source, the app still enforces a real, observable consequence: `bypass-permissions mode DISABLED by enterprise policy (fail-closed: policy could not be determined)`. Confirms the resolver has real teeth, not just logging.

Diagram 2 — What we hope to confirm, to make this work end to end

This is the fleet-wide, no-per-device-DB-write version of the same idea (X2) — routing control, not model-access removal (the real model's answer still flows back untouched). Every arrow here is **design intent**: architecturally plausible from what's confirmed in Diagram 1 and GitHub's own docs, but none of it has been built or observed working end to end yet.

-- Unconfirmed — needs verification



What each arrow means

1 Pushes managed-settings via native MDM unconfirmed

An **alternative path to the console entirely**: IT pushes a Workspace ONE/Jamf/Intune configuration profile (macOS Managed Preferences domain `com.github.copilot`, or the Windows registry path), or simply drops a `managed-settings.json` file at the fixed OS path. This never touches GitHub's backend for this tier. **Unconfirmed** whether the model-availability policy specifically can be delivered this way, versus only simpler scalar keys like `model`.

2 Registers custom model + URL unconfirmed

Enterprise admin console → AI controls → Copilot → Configure custom models → Add API key, selecting the "OpenAI-compatible" provider type. **Unconfirmed**: GitHub's own docs describe a URL field for Microsoft Foundry but never describe one for OpenAI-compatible, despite listing it as supported — a real, verbatim-confirmed documentation gap, not yet resolved by hands-on testing.

3 Disables GitHub-hosted models unconfirmed

Admin sets every GitHub-hosted model to Disabled and sets "Default availability for released models" so new ones don't auto-enable. **Unconfirmed:** no doc states what a user sees if every built-in model is off and only the custom one remains — the "zero built-in models" end state has never been directly observed.

4 Reads native MDM policy (now present) unconfirmed

Unlike Diagram 1 (where this check returned "none present"), a managed device would have a real MDM-delivered policy here. Per GitHub's own documented precedence, **native MDM wins outright over the server-managed tier** — so if this path works, it doesn't need the console/self-fetch path (steps 2, 3, 5) to work at all. **Unconfirmed** whether model-availability policy is actually one of the keys deliverable this way.

5 Delivers policy via self-fetch unconfirmed

The console-configured policy reaching the app through the same `managed_settings::api_session` self-fetch mechanism proven live in Diagram 1. **Unconfirmed:** that self-fetch has only ever been observed returning "none" for an unmanaged personal account — never observed carrying a real, non-empty admin-configured policy. A real, related bug report (GitHub issue `copilot-cli#4959`) shows the app's `model` managed-setting being fetched correctly but not applied — a known reliability problem on this exact tier.

6 Resolves: MDM > server > file unconfirmed

The app would need to resolve whichever source actually won (MDM, if present; otherwise server; otherwise the local file) into "this custom model is the default and only selectable entry."
Unconfirmed — this resolution has never been observed with a real policy present from any tier.

7 Inference call redirected here unconfirmed

The real inference call would go to the enterprise-registered endpoint instead of GitHub's own CAPI backend. **Unconfirmed** whether GitHub's own infrastructure actually honors this redirect for an OpenAI-compatible provider specifically, given the URL-field gap in item 2.

8 Forwards request not yet built

The registered endpoint would need to relay the untouched request to Optimus. **Not yet built** for this app — no code exists for this hop; it's the same "forward and observe" pattern already used elsewhere in this codebase, not yet wired up here.

9 Forwards to real backend not yet built

Optimus, after enforcing policy, would relay the request on to GitHub's real Copilot backend. **Not yet built.**

10 Real model response not yet built

GitHub's real backend would generate a genuine completion and return it to Optimus. **Not yet built** — depends on item 7 actually reaching a real backend in the first place.

11 Relays response back not yet built

Optimus would apply any response-side policy and relay the answer back toward the registered endpoint. **Not yet built.**

The endpoint relays the real answer back to the app, which shows it to the user exactly as Diagram 1's step 4 already proved. **Additionally unconfirmed:** the live test in Diagram 1 only covered a plain Chat — Project/agentic sessions showed a different, locked-in first-party model with no picker reaching a custom provider at all, so this entire chain may not apply there.